Supervised learning

```
# 1. Import libraries

import pandas as pd


from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix


# 2. Load dataset

uploaded = files.upload()

df = pd.read_csv(next(iter(uploaded)))


print("Dataset Shape:", df.shape)


# 3. Select input features and target

# Change these columns for another problem statement


X = df[['Age', 'MonthlyIncome', 'YearsAtCompany']].copy()


# Convert OverTime into numerical values

X['OverTime'] = df['OverTime'].map({'No': 0, 'Yes': 1})


# Convert target into numerical values

y = df['Attrition'].map({'No': 0, 'Yes': 1})


print("\nTarget Distribution:")

print(df['Attrition'].value_counts())
```

```
# 4. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining Records:", len(X_train))
print("Testing Records:", len(X_test))

# 5. Create and train Decision Tree model
model = DecisionTreeClassifier(max_depth=4, random_state=42)

model.fit(X_train, y_train)

# 6. Test the model
y_pred = model.predict(X_test)

# 7. Evaluate model
print("\nModel Evaluation")

print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_test, y_pred, zero_division=0) * 100, 2), "%")
print("Recall:", round(recall_score(y_test, y_pred, zero_division=0) * 100, 2), "%")
print("F1 Score:", round(f1_score(y_test, y_pred, zero_division=0) * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 8. Predict output for a new employee
# Values: Age, MonthlyIncome, YearsAtCompany, OverTime

new_employee = pd.DataFrame(
```

```
    [[30, 3000, 3, 1]],
    columns=X.columns
)

prediction = model.predict(new_employee)

print("\nPrediction for New Employee:")

if prediction[0] == 1:
    print("Attrition: Yes")
else:
    print("Attrition: No")
```

Algorithm	**When to choose**	**Example**
Linear Regression	Output is continuous	Predict salary
Logistic Regression	Binary or multiclass classification	Predict attrition
KNN	Similar data points tend to have similar outputs	Predict pass/fail
Decision Tree	Decisions can be made using conditions	Predict loan approval
SVM	Classification based on separating boundaries	Predict disease category

Basic dataset information

```
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
```

```
print("\nMissing Values:")
print(df.isnull().sum())

print("\nAvailable Columns:")
print(df.columns.tolist())
```

Correlation analysis

```
import matplotlib.pyplot as plt

# Convert target into numerical values
df['Attrition_num'] = df['Attrition'].map({'No': 0, 'Yes': 1})

# Find correlation of numerical features with target
corr = df.select_dtypes(include='number').corr()['Attrition_num']

# Display strongest relationships
print(corr.drop('Attrition_num').dropna()
    .sort_values(key=abs, ascending=False))
```

Preprocessing

```
# 1. Separate input features and target
X = df[['Feature1', 'Feature2', 'Feature3']].copy()
y = df['Target']

# 2. Split data BEFORE preprocessing
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
```

```
# 3. Identify numerical and categorical columns
num_cols = X_train.select_dtypes(include='number').columns
cat_cols = X_train.select_dtypes(exclude='number').columns

# 4. Handle missing numerical values
for col in num_cols:
    median = X_train[col].median()
    X_train[col] = X_train[col].fillna(median)
    X_test[col] = X_test[col].fillna(median)

# 5. Handle missing categorical values
for col in cat_cols:
    mode = X_train[col].mode()
    value = mode[0] if len(mode) > 0 else 'Unknown'
    X_train[col] = X_train[col].fillna(value)
    X_test[col] = X_test[col].fillna(value)

# 6. Convert categorical values into numerical values
X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)

# Ensure both datasets have identical columns
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

# Feature scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3. Simple code changes for different algorithms

The best part is that you don't need to change your entire implementation.

Only change the model declaration.

A. Classification algorithms

1. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
```

2. Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
```

3. K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
```

4. Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
```

5. Naive Bayes

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
```

6. Support Vector Machine (SVM)

```
from sklearn.svm import SVC

model = SVC()
```

B. Regression algorithms

1. Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
```

What remains unchanged?

For all these algorithms, the training and prediction statements are the same:

```
# Training

model.fit(X_train, y_train)
```

Testing

y_pred = model.predict(X_test)

Predicting unseen data

new_prediction = model.predict(new_data)

Only the model declaration and suitable evaluation metrics need to change.

4. Evaluation metrics: What to use?

Classification	**Regression**
Accuracy	MAE
Precision	MSE
Recall	RMSE
F1 Score	R^2 Score
Confusion Matrix	Actual vs Predicted

Metric	**Meaning**	**Better value**
MAE	Average absolute prediction error	Lower
MSE	Average squared prediction error	Lower
RMSE	Square root of MSE	Lower
R^2 Score	How well the model explains variation in target	Closer to 1

from sklearn.metrics import *

Change according to problem statement

problem_type = "classification"

print("MODEL EVALUATION SUMMARY")

if problem_type == "classification":

```
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred, average='weighted', zero_division=0))
    print("Recall:", recall_score(y_test, y_pred, average='weighted', zero_division=0))
    print("F1 Score:", f1_score(y_test, y_pred, average='weighted', zero_division=0))

    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))

elif problem_type == "regression":

    print("MAE:", mean_absolute_error(y_test, y_pred))
    print("MSE:", mean_squared_error(y_test, y_pred))
    print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)
    print("R2 Score:", r2_score(y_test, y_pred))
```

EXP2

```
# ========================================
# UNSUPERVISED LEARNING: K-MEANS + HIERARCHICAL
# ========================================

# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score
from sklearn.decomposition import PCA
```

```
from scipy.cluster.hierarchy import linkage, dendrogram

# ========================================
# 2. LOAD DATASET
# ========================================

from google.colab import files
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

# CHANGE THESE TWO VALUES FOR ANOTHER DATASET
TARGET_COL = 'Species'      # Use None if no actual label exists
DROP_COLS = []              # Remove ID or irrelevant columns if required

print("Dimensions:", df.shape)
print("\nColumns:\n", df.columns)
print("\nData Types:\n", df.dtypes)
print("\nFirst 5 Records:\n", df.head())
print("\nStatistics:\n", df.describe())

# ========================================
# 3. DATA CLEANING
# ========================================

print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())

# Remove duplicate records
df = df.drop_duplicates().reset_index(drop=True)

# Separate actual labels (never use them for clustering)
```

```
if TARGET_COL in df.columns:
   y = df[TARGET_COL].copy()
else:
   y = None

# Select only numerical input features
X = df.drop(columns=[TARGET_COL] + DROP_COLS, errors='ignore')
X = X.select_dtypes(include=np.number)

print("\nSelected Features:\n", X.columns.tolist())

# Outlier analysis using IQR
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)
IQR = Q3 - Q1

outliers = ((X < Q1 - 1.5 * IQR) |
        (X > Q3 + 1.5 * IQR)).sum()

print("\nOutlier Count:\n", outliers)

# Keep genuine observations; do not automatically remove outliers.

# ========================================
# 4. EXPLORATORY DATA ANALYSIS (EDA)
# ========================================


# Feature distributions
X.hist(figsize=(10, 7))
plt.tight_layout()
```

```
plt.show()

# Box plots
X.plot(kind='box', subplots=True, layout=(2, 2),
    figsize=(10, 6))
plt.tight_layout()
plt.show()

# Correlation heatmap
sns.heatmap(X.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation")
plt.show()

# Pairwise analysis (first 5 features)
sns.pairplot(X.iloc[:, :5])
plt.show()

# ========================================
# 5. SPLIT AND PREPROCESS DATA
# ========================================

# Keep 10 observations aside for new-data prediction
X_train = X.iloc[:-10].copy()
X_new = X.iloc[-10:].copy()

# Fill missing values using training data medians
medians = X_train.median()
X_train = X_train.fillna(medians)
X_new = X_new.fillna(medians)

# Standardization
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
X_new_scaled = scaler.transform(X_new)

print("\nTraining Records:", len(X_train))
print("New Observations:", len(X_new))

# ========================================
# 6. FIND OPTIMAL K (K-MEANS)
# ========================================

inertia = []
silhouette = []
K_values = range(2, 7)

for k in K_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)

    inertia.append(km.inertia_)
    silhouette.append(silhouette_score(X_scaled, labels))

# Elbow and Silhouette plots
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(K_values, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
```

```
plt.subplot(1, 2, 2)
plt.plot(K_values, silhouette, marker='o')
plt.title("Silhouette Score")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Score")

plt.tight_layout()
plt.show()

# Select K with highest Silhouette Score
best_k = list(K_values)[np.argmax(silhouette)]
print("\nSelected K:", best_k)

# ========================================
# 7. K-MEANS CLUSTERING
# ========================================

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
k_labels = kmeans.fit_predict(X_scaled)

print("\nK-Means Cluster Sizes:")
print(pd.Series(k_labels).value_counts().sort_index())

print("\nK-Means Centroids (Original Scale):")
print(pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=X.columns
))

print("\nInertia:", kmeans.inertia_)
```

```
print("Silhouette Score:", silhouette_score(X_scaled, k_labels))

# ========================================
# 8. HIERARCHICAL CLUSTERING
# ========================================

# Compare three linkage methods
linkages = ['ward', 'complete', 'average']
best_linkage = None
best_score = -1

for method in linkages:
    Z = linkage(X_scaled, method=method)

    h_model = AgglomerativeClustering(
        n_clusters=best_k, linkage=method
    )

    h_labels = h_model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, h_labels)

    print(method, "Silhouette Score:", round(score, 4))

    if score > best_score:
        best_score = score
        best_linkage = method

print("\nBest Linkage:", best_linkage)

# Dendrogram (maximum 200 observations for readability)
Z = linkage(X_scaled[:200], method=best_linkage)
```

```
plt.figure(figsize=(12, 5))
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Observations")
plt.ylabel("Distance")
plt.show()

# Final hierarchical clusters
hierarchy = AgglomerativeClustering(
   n_clusters=best_k, linkage=best_linkage
)

h_labels = hierarchy.fit_predict(X_scaled)

print("\nHierarchical Cluster Sizes:")
print(pd.Series(h_labels).value_counts().sort_index())

print("Hierarchical Silhouette:",
    silhouette_score(X_scaled, h_labels))

# ========================================
# 9. PCA VISUALIZATION
# ========================================

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("\nPC1 Variance:", pca.explained_variance_ratio_[0])
print("PC2 Variance:", pca.explained_variance_ratio_[1])
print("Total Variance Explained:",
```

```
    sum(pca.explained_variance_ratio_))

plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=k_labels, cmap='viridis')
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("K-Means Clusters using PCA")
plt.show()

# ========================================
# 10. EXTERNAL EVALUATION
# ========================================

if y is not None:
   y_train = y.iloc[:-10]

   print("\nCluster vs Actual Species:")
   print(pd.crosstab(k_labels, y_train))

   print("\nARI:", adjusted_rand_score(y_train, k_labels))
   print("NMI:", normalized_mutual_info_score(y_train, k_labels))

else:
   print("\nNo actual labels available for external evaluation.")

# ========================================
# 11. CLUSTER PROFILES
# ========================================

profile = X_train.copy()
profile['Cluster'] = k_labels
```

```
print("\nCluster Profiles:")
print(profile.groupby('Cluster').mean())

# ========================================
# 12. PREDICT NEW OBSERVATIONS
# ========================================

new_labels = kmeans.predict(X_new_scaled)

result = X_new.copy()
result['Assigned Cluster'] = new_labels

print("\nNew Observations and Assigned Clusters:")
print(result)
#============================================
import pandas as pd
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Convert into DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add actual species labels
df['Species'] = iris.target

# Display dataset
print(df.head())
print("Shape:", df.shape)
```

Associative Memory Networks

```
import numpy as np

# ========================================
# 1. SELECT NETWORK
# ========================================

MODE = "auto"   # auto / hetero / bam / hopfield

# ========================================
# 2. ENTER PATTERNS (CHANGE THESE VALUES)
# ========================================

if MODE == "auto":

    A = np.array([
        [1, 1, 1, -1, -1, -1],
        [-1, -1, 1, 1, 1, -1],
        [1, -1, -1, -1, 1, 1],
        [-1, 1, -1, 1, -1, 1]
    ])

    test = np.array([1, 1, -1, -1, -1, -1])

elif MODE == "hetero":

    A = np.array([
        [1, 1, -1, -1],
```

```
    [-1, -1, 1, 1],
    [1, -1, 1, -1],
    [-1, 1, -1, 1]
  ])

  B = np.array([
    [1, 1, -1],
    [-1, -1, 1],
    [1, -1, -1],
    [-1, 1, 1]
  ])

elif MODE == "bam":

  A = np.array([
    [1, 1, 1, 1, 1, 1],
    [-1, -1, -1, -1, -1, -1],
    [1, 1, -1, -1, 1, 1],
    [-1, -1, 1, 1, -1, -1]
  ])

  B = np.array([
    [1, 1, 1],
    [-1, -1, -1],
    [1, -1, 1],
    [-1, 1, -1]
  ])

elif MODE == "hopfield":

  A = np.array([
```

```
    [1, 1, 1, -1, -1, -1],
    [-1, -1, -1, 1, 1, 1]
  ])

  test = np.array([1, -1, 1, -1, -1, -1])


# Bipolar activation function
def sign(x):
  return np.where(x >= 0, 1, -1)


# ======================================
# 3. CALCULATE WEIGHT MATRIX
# ======================================

if MODE == "hetero" or MODE == "bam":
  W = A.T @ B

else:
  W = A.T @ A
  np.fill_diagonal(W, 0)

print("Weight Matrix:\n", W)


# ======================================
# 4. AUTOASSOCIATIVE MEMORY
# ======================================

if MODE == "auto":
```

```
    x = test.copy()

    # Asynchronous recall until stable
    for epoch in range(20):
        old = x.copy()

        for i in range(len(x)):
            net = W[i] @ x

            if net > 0:
                x[i] = 1
            elif net < 0:
                x[i] = -1

        if np.array_equal(old, x):
            break

    print("\nNoisy Input:", test)
    print("Recalled Pattern:", x)



# ======================================
# 5. HETEROASSOCIATIVE MEMORY
# ======================================

elif MODE == "hetero":

    for i in range(len(A)):
        output = sign(A[i] @ W)
```

```
    print("\nInput:", A[i])
    print("Target:", B[i])
    print("Output:", output)


# ======================================
# 6. BAM
# ======================================

elif MODE == "bam":

  for i in range(len(A)):

    # Forward: A -> B
    x = A[i].copy()
    y = np.zeros(len(B[i]), dtype=int)

    for _ in range(20):
      y_new = sign(x @ W)
      x_new = sign(y_new @ W.T)

      if np.array_equal(x, x_new) and np.array_equal(y, y_new):
        break

      x, y = x_new, y_new

    print("\nA -> B:", A[i], "->", y)

    # Backward: B -> A
    y = B[i].copy()
    x = np.zeros(len(A[i]), dtype=int)
```

```
    for _ in range(20):
        x_new = sign(y @ W.T)
        y_new = sign(x_new @ W)

        if np.array_equal(x, x_new) and np.array_equal(y, y_new):
            break

        x, y = x_new, y_new

    print("B -> A:", B[i], "->", x)



# ========================================
# 7. HOPFIELD NETWORK
# ========================================

elif MODE == "hopfield":

    x = test.copy()

    # Update one neuron at a time
    for epoch in range(20):

        old = x.copy()

        for i in range(len(x)):
            net = W[i] @ x

            if net > 0:
                x[i] = 1
```

```
        elif net < 0:
            x[i] = -1

        # Energy calculation
        E = -0.5 * x @ W @ x
        print("Energy:", E)

    if np.array_equal(old, x):
        break

print("\nNoisy Input:", test)
print("Stable Pattern:", x)

print("Stored Pattern Matched:",
      any(np.array_equal(x, p) for p in A))
```

Fuzzy sets exp 8

```
# ================================================
# AISC EXPERIMENT: FUZZY LOGIC
# SMART GREENHOUSE CONTROLLER
# ================================================

import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(precision=3, suppress=True)

# ================================================
# 1. INPUT DATA (CHANGE THIS SECTION)
```

```
# ================================================

X = np.array([10, 15, 20, 25, 30, 35, 40])

T = np.array([15, 20, 25, 30, 35])

# Membership function parameters
Cold_params = (5, 10, 15, 25)
Warm_params = (15, 25, 35)
Hot_params = (25, 35, 40, 45)
Comfort_params = (25, 5)

# Fuzzy relations
R1 = np.array([
   [0.2, 0.5, 0.7, 0.3],
   [0.7, 0.6, 0.2, 0.1],
   [1.0, 0.4, 0.1, 0.0],
   [0.3, 0.6, 0.8, 0.4],
   [0.0, 0.2, 0.6, 0.9]
])

R2 = np.array([
   [0.1, 0.0, 0.0, 0.1],
   [0.5, 0.3, 0.2, 0.2],
   [0.6, 0.7, 0.5, 0.4],
   [0.4, 0.6, 0.9, 0.2]
])

R3 = np.array([
   [0.8, 0.4, 0.1],
   [0.5, 0.7, 0.3],
```

```
  [0.6, 0.5, 0.2],
  [0.1, 0.4, 0.9]
])

Q = np.array([
  [1.0, 0.8, 0.4, 0.2, 0.1],
  [0.8, 1.0, 0.9, 0.3, 0.2],
  [0.4, 0.9, 1.0, 0.5, 0.3],
  [0.2, 0.3, 0.5, 1.0, 0.7],
  [0.1, 0.2, 0.3, 0.7, 1.0]
])

Stress = ['None', 'Low', 'Medium', 'High']
Actions = ['Ventilate', 'Mist', 'Shade', 'Heat']
Energy = ['Low', 'Medium', 'High']

# ================================================
# 2. MEMBERSHIP FUNCTIONS
# ================================================

def triangle(x, a, b, c):
  return np.maximum(0, np.minimum(
    (x-a)/(b-a), (c-x)/(c-b)
  ))

def trapezoid(x, a, b, c, d):
  return np.maximum(0, np.minimum(
    np.minimum((x-a)/(b-a), 1),
    (d-x)/(d-c)
  ))
```

```
def gaussian(x, mean, sigma):
    return np.exp(-((x-mean)**2)/(2*sigma**2))



Cold = trapezoid(X, *Cold_params)
Warm = triangle(X, *Warm_params)
Hot = trapezoid(X, *Hot_params)
Comfortable = gaussian(X, *Comfort_params)

sets = {
    'Cold': Cold,
    'Warm': Warm,
    'Hot': Hot,
    'Comfortable': Comfortable
}

print("FUZZY SETS")
for name, value in sets.items():
    print(name, ":", value)

# ================================================
# A. CRISP VS FUZZY LOGIC
# ================================================

print("\nA. CRISP VS FUZZY")

for temp in [29.9, 30.1]:
    crisp = 1 if temp >= 30 else 0
    fuzzy = trapezoid(np.array([temp]), *Hot_params)[0]

    print(temp, "Crisp:", crisp, "Fuzzy:", round(fuzzy, 2))
```

```
print("Crisp Jump: 1")
print("Fuzzy Jump: 0.02")

# ================================================
# B. MEMBERSHIP FUNCTION PLOT
# ================================================

plt.figure(figsize=(8, 5))

for name, value in sets.items():
    plt.plot(X, value, marker='o', label=name)

plt.title("Fuzzy Membership Functions")
plt.xlabel("Temperature")
plt.ylabel("Membership")
plt.legend()
plt.grid()
plt.show()

# Support, Core, Height, Cardinality, Alpha cuts
print("\nB. FUZZY SET PROPERTIES")

for name in ['Warm', 'Comfortable']:
    mu = sets[name]

    print("\n", name)
    print("Support:", X[mu > 0])
    print("Core:", X[np.isclose(mu, 1)])
    print("Height:", np.max(mu))
    print("Cardinality:", np.sum(mu))
```

```
    print("Normal:", np.isclose(np.max(mu), 1))

    convex = all(
        mu[i] >= min(mu[i-1], mu[i+1]) - 1e-9
        for i in range(1, len(mu)-1)
    )
    print("Convex:", convex)

    for alpha in [0.25, 0.5, 0.75, 1]:
        print(alpha, "-cut:", X[mu >= alpha])

# Fuzziness and entropy
print("\nFUZZINESS AND ENTROPY")

fuzziness = {}
entropy = {}

for name, mu in sets.items():

    # Normalized fuzziness index
    fuzziness[name] = np.mean(2 * np.minimum(mu, 1-mu))

    # Normalized Shannon fuzzy entropy
    p = np.clip(mu, 1e-12, 1-1e-12)
    entropy[name] = np.mean(
        -(p*np.log2(p) + (1-p)*np.log2(1-p))
    )

    print(name, "Fuzziness:", round(fuzziness[name], 4),
          "Entropy:", round(entropy[name], 4))
```

```
print("Fuzziness Ranking:",
      sorted(fuzziness, key=fuzziness.get, reverse=True))

# ================================================
# C. CLASSICAL SET OPERATIONS
# ================================================

print("\nC. CLASSICAL SET OPERATIONS")

universe = set(X)

classical = {
    'Cold': {10, 15, 20},
    'Warm': {20, 25, 30},
    'Hot': {30, 35, 40}
}

for name, value in classical.items():
    print(name, "Complement:", universe - value)

for a, b in [('Cold', 'Warm'), ('Warm', 'Hot'),
             ('Cold', 'Hot')]:

    A = classical[a]
    B = classical[b]

    print(a, "Union", b, ":", A | B)
    print(a, "Intersection", b, ":", A & B)
    print(a, "Difference", b, ":", A - B)

# ================================================
```

```
# 3. FUZZY SET OPERATIONS
# ================================================

print("\nFUZZY SET OPERATIONS")

A = Cold
B = Warm

print("Union:", np.maximum(A, B))
print("Intersection:", np.minimum(A, B))
print("Complement:", 1-A)
print("Difference:", np.minimum(A, 1-B))

print("Algebraic Product:", A*B)
print("Algebraic Sum:", A+B-A*B)
print("Bounded Sum:", np.minimum(1, A+B))
print("Bounded Difference:", np.maximum(0, A-B))

# Hamacher operators
den = A+B-A*B
hamacher_t = np.divide(
   A*B, den, out=np.zeros_like(A), where=den != 0
)

den = 1-A*B
hamacher_s = np.divide(
   A+B-2*A*B, den, out=np.ones_like(A), where=den != 0
)

# Yager operators, w = 2
yager_t = np.maximum(0, 1-np.sqrt((1-A)**2+(1-B)**2))
```

```
yager_s = np.minimum(1, np.sqrt(A**2+B**2))

# Einstein operators
einstein_t = A*B/(2-A-B+A*B)
einstein_s = (A+B)/(1+A*B)

print("\nAlternative Operators")
operators = {
   'Min-Max': (np.minimum(A, B), np.maximum(A, B)),
   'Hamacher': (hamacher_t, hamacher_s),
   'Yager': (yager_t, yager_s),
   'Einstein': (einstein_t, einstein_s)
}

for name, (t, s) in operators.items():
   print(name, "Intersection:", t)
   print(name, "Union:", s)

# Rank by average output (lower = more conservative)
print("\nIntersection Ranking (conservative to liberal):")
print(sorted(operators,
         key=lambda n: np.mean(operators[n][0])))

print("Union Ranking (conservative to liberal):")
print(sorted(operators,
         key=lambda n: np.mean(operators[n][1])))

# ================================================
# D. VERIFY FUZZY SET PROPERTIES
# ================================================
```

```
print("\nD. VERIFY FUZZY SET PROPERTIES")

def check_laws(tnorm, snorm, name):

  a, b, c = Cold, Warm, Hot
  not_a = 1-a
  not_b = 1-b

  laws = {
    'Commutativity':
    np.allclose(tnorm(a,b), tnorm(b,a)) and
    np.allclose(snorm(a,b), snorm(b,a)),

    'Associativity':
    np.allclose(tnorm(tnorm(a,b),c),
          tnorm(a,tnorm(b,c))) and
    np.allclose(snorm(snorm(a,b),c),
          snorm(a,snorm(b,c))),

    'Distributivity':
    np.allclose(tnorm(a,snorm(b,c)),
          snorm(tnorm(a,b),tnorm(a,c))) and
    np.allclose(snorm(a,tnorm(b,c)),
          tnorm(snorm(a,b),snorm(a,c))),

    'Idempotency':
    np.allclose(tnorm(a,a),a) and
    np.allclose(snorm(a,a),a),

    'Identity':
    np.allclose(tnorm(a,np.ones_like(a)),a) and
```

```
        np.allclose(snorm(a,np.zeros_like(a)),a),

        'Involution':
        np.allclose(1-(1-a),a),

        'De Morgan':
        np.allclose(1-snorm(a,b),tnorm(not_a,not_b)) and
        np.allclose(1-tnorm(a,b),snorm(not_a,not_b))
    }

    print("\n", name)
    for law, result in laws.items():
        print(law, ":", result)

# Standard fuzzy operators
check_laws(np.minimum, np.maximum, "MIN-MAX OPERATORS")

# Algebraic operators
check_laws(lambda x,y: x*y,
           lambda x,y: x+y-x*y,
           "ALGEBRAIC OPERATORS")

# Show excluded middle and contradiction violations
not_warm = 1-Warm
union = np.maximum(Warm, not_warm)
intersection = np.minimum(Warm, not_warm)

v1 = np.abs(1-union)
v2 = intersection

print("\nExcluded Middle Maximum Violation:", np.max(v1))
```

```
print("Occurs at:", X[np.isclose(v1, np.max(v1))])

print("Contradiction Maximum Violation:", np.max(v2))
print("Occurs at:", X[np.isclose(v2, np.max(v2))])

# Counterexamples for algebraic distributivity
a, b, c = 0.2, 0.6, 0.8
prod = lambda x,y: x*y
summ = lambda x,y: x+y-x*y

print("\nAlgebraic Distributivity Counterexamples:")
print("Product over Sum difference:",
    prod(a, summ(b,c))-summ(prod(a,b),prod(a,c)))

print("Sum over Product difference:",
    summ(a, prod(b,c))-prod(summ(a,b),summ(a,c)))

print("Product Idempotency difference:", prod(a,a)-a)
print("Sum Idempotency difference:", summ(a,a)-a)

# ================================================
# 4. DECOMPOSITION THEOREM
# ================================================

print("\nDECOMPOSITION THEOREM")

reconstructed = np.zeros_like(Warm)

for alpha in np.unique(Warm):
  if alpha > 0:
    reconstructed = np.maximum(
```

```
        reconstructed,
        alpha*(Warm >= alpha)
    )

print("Original Warm:", Warm)
print("Reconstructed:", reconstructed)
print("Verified:", np.allclose(Warm, reconstructed))

# ================================================
# E. FUZZY RELATIONS
# ================================================

print("\nE. FUZZY RELATIONS")

# Classical Cartesian product
zones = ['z1', 'z2']
actuators = ['Fan', 'Heater']

print("Classical Cartesian Product:",
    [(z,a) for z in zones for a in actuators])

# Fuzzy Cartesian product: Warm x Humid
Warm_T = triangle(T, *Warm_params)
Humid = np.array([0.2, 0.6, 1.0])

R = np.minimum(Warm_T[:, None], Humid[None, :])

print("\nFuzzy Cartesian Product:\n", R)
print("Projection on Temperature:", np.max(R, axis=1))
print("Projection on Humidity:", np.max(R, axis=0))
```

```
# ================================================
# 5. MAX-MIN AND MAX-PRODUCT COMPOSITION
# ================================================

def max_min(R, S):
    return np.max(
        np.minimum(R[:, :, None], S[None, :, :]),
        axis=1
    )

def max_product(R, S):
    return np.max(
        R[:, :, None]*S[None, :, :],
        axis=1
    )

def vector_comp(v, R):
    return np.max(np.minimum(v[:, None], R), axis=0)

# R1 o R2
R12_min = max_min(R1, R2)
R12_product = max_product(R1, R2)

print("\nR1 o R2 (Max-Min):\n", R12_min)
print("\nR1 o R2 (Max-Product):\n", R12_product)

# Full chain R1 o R2 o R3
R123_min = max_min(R12_min, R3)
R123_product = max_product(R12_product, R3)

print("\nFull Chain Max-Min:\n", R123_min)
```

```
print("\nFull Chain Max-Product:\n", R123_product)

# Associativity
left = max_min(max_min(R1, R2), R3)
right = max_min(R1, max_min(R2, R3))

left_p = max_product(max_product(R1, R2), R3)
right_p = max_product(R1, max_product(R2, R3))

print("\nMax-Min Associativity:", np.allclose(left, right))
print("Max-Product Associativity:", np.allclose(left_p, right_p))

# ================================================
# 6. NON-COMMUTATIVITY
# ================================================

P = np.array([[0.9, 0.2],
              [0.4, 0.7]])

S = np.array([[0.5, 0.8],
              [0.1, 0.6]])

print("\nP o S:\n", max_min(P, S))
print("S o P:\n", max_min(S, P))
print("Commutative:", np.allclose(max_min(P,S), max_min(S,P)))

print("\nR2 o R1 is undefined because:")
print("R2 dimensions:", R2.shape)
print("R1 dimensions:", R1.shape)
print("Inner dimensions 4 and 5 do not match.")
```

```
# ================================================
# 7. DISTRIBUTIVITY OF COMPOSITION
# ================================================

R_demo = np.array([[0.8, 0.4]])
S_demo = np.array([[0.9, 0.1],
                   [0.2, 0.7]])
U_demo = np.array([[0.3, 0.6],
                   [0.5, 0.8]])

left = max_min(R_demo, np.maximum(S_demo, U_demo))
right = np.maximum(max_min(R_demo,S_demo),
                   max_min(R_demo,U_demo))

print("\nComposition over Union:", np.allclose(left,right))

# Strict intersection inclusion example
R_demo = np.array([[1.0, 1.0]])
S_demo = np.array([[1.0], [0.0]])
U_demo = np.array([[0.0], [1.0]])

left = max_min(R_demo, np.minimum(S_demo,U_demo))
right = np.minimum(max_min(R_demo,S_demo),
                   max_min(R_demo,U_demo))

print("R o (S intersection U):", left)
print("(R o S) intersection (R o U):", right)
print("Strict Inclusion:", np.all(left <= right + 1e-9))

# ================================================
# 8. FUZZY READING AND DEFUZZIFICATION
```

```
# ================================================

reading = np.array([0.0, 0.2, 0.7, 1.0, 0.0])

action_mu = vector_comp(reading, R12_min)

print("\nFuzzy Input:", reading)
print("Action Membership:", action_mu)

# Centroid using ordinal action indices 0,1,2,3
indices = np.arange(len(Actions))
centroid = np.sum(indices*action_mu)/np.sum(action_mu)

centroid_action = Actions[np.argmin(np.abs(indices-centroid))]
max_action = Actions[np.argmax(action_mu)]

print("Centroid:", centroid)
print("Centroid Action:", centroid_action)
print("Max Membership Action:", max_action)

# Crisp reading at 30 degrees
crisp_reading = np.array([0, 0, 0, 1, 0])
crisp_action_mu = vector_comp(crisp_reading, R12_min)

print("Crisp Action Membership:", crisp_action_mu)
print("Crisp Action:", Actions[np.argmax(crisp_action_mu)])

# ================================================
# F. TOLERANCE AND EQUIVALENCE RELATIONS
# ================================================
```

```
print("\nF. TOLERANCE RELATIONS")

print("Reflexive:", np.allclose(np.diag(Q), 1))
print("Symmetric:", np.allclose(Q, Q.T))

print("Max-Min Transitive:",
    np.all(max_min(Q,Q) <= Q + 1e-9))

# Max-min transitive closure
closure = Q.copy()
iterations = 0

while True:
  new = np.maximum(closure, max_min(closure, closure))
  iterations += 1

  if np.allclose(new, closure):
    break

  closure = new

print("\nTransitive Closure:\n", closure)
print("Iterations:", iterations)

# Equivalence classes using alpha cuts
print("\nTemperature Partitions")

for alpha in [0.9, 0.8, 0.7, 0.5]:

  unused = set(range(len(T)))
  groups = []
```

```
    while unused:
        i = min(unused)

        group = [
            j for j in sorted(unused)
            if closure[i,j] >= alpha
        ]

        groups.append([T[j] for j in group])
        unused.difference_update(group)

    print(alpha, "-cut:", groups)

# ================================================
# G. EXTENSION PRINCIPLE
# ================================================

print("\nG. EXTENSION PRINCIPLE")

# Single-variable mapping y = d^2 - 2d
D = [-2, -1, 0, 1, 2]
D_mu = [0.3, 0.6, 1.0, 0.8, 0.5]

result = {}

for d, mu in zip(D, D_mu):
    y = d**2 - 2*d
    result[y] = max(result.get(y, 0), mu)

print("Single Variable Result:")
```

```
print(dict(sorted(result.items())))

# Two-variable mapping z = a*b
values_A = [1, 2, 3]
mu_A = [0.5, 1.0, 0.6]

values_B = [2, 3, 4]
mu_B = [0.4, 1.0, 0.7]

result = {}

for a, ma in zip(values_A, mu_A):
   for b, mb in zip(values_B, mu_B):

      z = a*b
      mu = min(ma, mb)

      result[z] = max(result.get(z, 0), mu)

print("\nTwo Variable Result:")
print(dict(sorted(result.items())))

# ================================================
# 9. ALPHA CUTS OF FUZZY NUMBERS
# ================================================

print("\nALPHA CUT INTERVALS")

A_params = (1, 2, 4)   # About 2
B_params = (2, 3, 5)   # About 3
```

```
def alpha_cut(params, alpha):
    a, b, c = params
    return (a+alpha*(b-a), c-alpha*(c-b))

for alpha in [0, 0.25, 0.5, 0.75, 1]:

    a1, a2 = alpha_cut(A_params, alpha)
    b1, b2 = alpha_cut(B_params, alpha)

    sum_interval = (a1+b1, a2+b2)
    product_interval = (a1*b1, a2*b2)

    print("\nAlpha:", alpha)
    print("A:", (a1,a2), "B:", (b1,b2))
    print("A+B:", sum_interval)
    print("A*B:", product_interval)

# ================================================
# 10. DIRECT EXTENSION PRINCIPLE VS ALPHA CUTS
# ================================================

# Discretized input universes
grid_A = np.round(np.arange(1, 4.001, 0.05), 3)
grid_B = np.round(np.arange(2, 5.001, 0.05), 3)

mu_A = triangle(grid_A, *A_params)
mu_B = triangle(grid_B, *B_params)

direct_sum = {}
direct_product = {}
```

```
# Direct extension principle:
# min memberships for each pair, max for collisions
for a, ma in zip(grid_A, mu_A):
  for b, mb in zip(grid_B, mu_B):

    membership = min(ma, mb)

    z = round(a+b, 4)
    direct_sum[z] = max(direct_sum.get(z,0), membership)

    z = round(a*b, 4)
    direct_product[z] = max(
      direct_product.get(z,0), membership
    )

# Membership obtained from alpha-cut intervals
def alpha_membership(z, operation):

  best = 0

  for alpha in np.linspace(0, 1, 201):

    a1, a2 = alpha_cut(A_params, alpha)
    b1, b2 = alpha_cut(B_params, alpha)

    if operation == 'sum':
      low, high = a1+b1, a2+b2
    else:
      low, high = a1*b1, a2*b2

    if low-1e-9 <= z <= high+1e-9:
```

```
            best = alpha

    return best

# Compare both methods
sum_error = max(
    abs(mu-alpha_membership(z,'sum'))
    for z,mu in direct_sum.items()
)

product_error = max(
    abs(mu-alpha_membership(z,'product'))
    for z,mu in direct_product.items()
)

print("\nMaximum Error")
print("Addition:", round(sum_error,4))
print("Multiplication:", round(product_error,4))

# Plot comparison
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
zs = sorted(direct_sum)
plt.plot(zs, [direct_sum[z] for z in zs], label='Direct')
plt.plot(zs, [alpha_membership(z,'sum') for z in zs],
         '--', label='Alpha Cut')
plt.title("Extension Principle: Addition")
plt.xlabel("Output")
plt.ylabel("Membership")
plt.legend()
```

```
plt.grid()


plt.subplot(1,2,2)
zs = sorted(direct_product)
plt.plot(zs, [direct_product[z] for z in zs], label='Direct')
plt.plot(zs, [alpha_membership(z,'product') for z in zs],
        '--', label='Alpha Cut')
plt.title("Extension Principle: Multiplication")
plt.xlabel("Output")
plt.ylabel("Membership")
plt.legend()
plt.grid()


plt.tight_layout()
plt.show()


print("\nEXPERIMENT COMPLETED")
```

Mamdani fuzzy controller

```
# ========================================
# MAMDANI FUZZY LOGIC CONTROLLER
# ========================================


import numpy as np
import matplotlib.pyplot as plt


# 1. Membership function (triangular)
def triangle(x, a, b, c):
    if x < a or x > c:
```

```
        return 0

    if x <= b:
        return 1 if a == b else (x-a)/(b-a)

    return 1 if b == c else (c-x)/(c-b)



# 2. Membership functions for inputs
# N = Negative, Z = Zero, P = Positive

error_mf = {
    'N': (-10, -10, 0),
    'Z': (-10, 0, 10),
    'P': (0, 10, 10)
}

change_mf = {
    'N': (-5, -5, 0),
    'Z': (-5, 0, 5),
    'P': (0, 5, 5)
}

# 3. Output membership functions
output_mf = {
    'Cooling': (-100, -100, 0),
    'Maintain': (-100, 0, 100),
    'Heating': (0, 100, 100)
}

# 4. Nine IF-THEN rules
```

```
rules = [
    ('N', 'N', 'Cooling'),
    ('N', 'Z', 'Cooling'),
    ('N', 'P', 'Maintain'),
    ('Z', 'N', 'Cooling'),
    ('Z', 'Z', 'Maintain'),
    ('Z', 'P', 'Heating'),
    ('P', 'N', 'Maintain'),
    ('P', 'Z', 'Heating'),
    ('P', 'P', 'Heating')
]

# 5. Output universe
U = np.linspace(-100, 100, 1001)

# Calculate output membership curves
out_curves = {}

for name, params in output_mf.items():
    out_curves[name] = np.array(
        [triangle(u, *params) for u in U]
    )

# 6. Mamdani inference and centroid defuzzification
def controller(e, de):

    # Fuzzification
    me = {
        name: triangle(e, *params)
        for name, params in error_mf.items()
    }
```

```
md = {
   name: triangle(de, *params)
   for name, params in change_mf.items()
}

# Aggregated output
aggregate = np.zeros(len(U))

# Apply all nine rules
for el, dl, out in rules:

   # Minimum = rule firing strength
   strength = min(me[el], md[dl])

   # Minimum = implication, maximum = aggregation
   clipped = np.minimum(
      strength, out_curves[out]
   )

   aggregate = np.maximum(aggregate, clipped)

# Centroid defuzzification
if np.sum(aggregate) == 0:
   return 0

result = np.sum(U * aggregate) / np.sum(aggregate)

return result
```

```
# 7. Calculate error from actual temperature
desired = 24
actual = 32

e = desired - actual

print("Desired Temperature:", desired)
print("Actual Temperature:", actual)
print("Temperature Error:", e)

# Change in error = current error - previous error
previous_error = -6
de = e - previous_error

print("Change in Error:", de)
print("Control Output:", round(controller(e, de), 2))



# 8. Test the five given cases
test_cases = [
    (8, 0),
    (3, 0),
    (0, 0),
    (-3, 0),
    (-8, 0)
]

print("\nTEST RESULTS")

for i, (e, de) in enumerate(test_cases, 1):
```

```
    result = controller(e, de)

    if result > 0:
        action = "Heating"
    elif result < 0:
        action = "Cooling"
    else:
        action = "Maintain"

    print("Case", i,
          "| Error:", e,
          "| Change:", de,
          "| Output:", round(result, 2),
          "| Action:", action)


# 9. Plot membership functions
plt.figure(figsize=(12, 4))

# Error membership functions
plt.subplot(1, 3, 1)
x = np.linspace(-10, 10, 200)

for name, params in error_mf.items():
    plt.plot(x, [triangle(v, *params) for v in x],
             label=name)

plt.title("Temperature Error")
plt.legend()
plt.grid()
```

```
# Change in error membership functions
plt.subplot(1, 3, 2)
x = np.linspace(-5, 5, 200)

for name, params in change_mf.items():
    plt.plot(x, [triangle(v, *params) for v in x],
             label=name)

plt.title("Change in Error")
plt.legend()
plt.grid()

# Output membership functions
plt.subplot(1, 3, 3)

for name, params in output_mf.items():
    plt.plot(U, out_curves[name], label=name)

plt.title("Control Output")
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

ANFIS SUEGENO FIRST ORDER

```
# ============================================
# ANFIS: TRAFFIC CONGESTION PREDICTION
# First-order Sugeno + Hybrid Learnin
```

```
# ============================================

import numpy as np
import matplotlib.pyplot as plt
from itertools import product

np.random.seed(42)

# ============================================
# 1. GENERATE DATASET (1500 RECORDS)
# ============================================

n = 1500

D = np.random.uniform(0, 100, n)  # Density
S = np.random.uniform(0, 80, n)    # Speed
T = np.random.uniform(0, 180, n)  # Waiting time

# Reference congestion score from the given formula
W = 100 * T / 180
Vc = 100 * (1 - S / 80)

# Small simulated noise to represent variation
C = np.clip(
    0.45*D + 0.35*W + 0.20*Vc +
    np.random.normal(0, 2, n),
    0, 100
)

print("Dataset Shape:", (n, 4))
print("\nFirst 5 Records:")
```

```
print(np.round(np.column_stack((D,S,T,C))[:5], 2))

# ================================================
# 2. PREPROCESSING
# ================================================

# Normalize inputs using their known ranges
X = np.column_stack((D/100, S/80, T/180))
y = C

# 80:20 train-test split
idx = np.random.permutation(n)
split = int(0.8*n)

train = idx[:split]
test = idx[split:]

X_train, X_test = X[train], X[test]
y_train, y_test = y[train], y[test]

print("\nTraining Records:", len(X_train))
print("Testing Records:", len(X_test))

# ================================================
# 3. INITIAL MEMBERSHIP FUNCTIONS
# ================================================

# Three Gaussian MFs for each input
centers = np.tile([0.0, 0.5, 1.0], (3,1))
sigmas = np.full((3,3), 0.25)
```

```
# Generate all 27 combinations of fuzzy sets
rules = list(product(range(3), repeat=3))
nr = len(rules)

# ============================================
# 4. FUZZY RULE FIRING STRENGTHS
# ============================================

def firing_weights(X, centers, sigmas):

  # Membership values of each input
  mu = [
    np.exp(-0.5*((X[:,j,None]-centers[j])
          / sigmas[j])**2)
    for j in range(3)
  ]

  # Product of memberships for each rule
  fire = np.ones((len(X), nr))

  for r, (i,j,k) in enumerate(rules):
    fire[:,r] = mu[0][:,i]*mu[1][:,j]*mu[2][:,k]

  # Normalize firing strengths
  return fire / (fire.sum(axis=1, keepdims=True)+1e-12)


# ============================================
# 5. SUGENO CONSEQUENT LAYER
# ============================================
```

```
def design_matrix(X, weights):

    # Each rule has p*x + q*y + r*z + s
    base = np.column_stack((X, np.ones(len(X))))

    return np.concatenate(
        [weights[:,i,None]*base for i in range(nr)],
        axis=1
    )



# ============================================
# 6. HYBRID LEARNING
# ============================================

epochs = 25
lr = 0.001
errors = []

for epoch in range(epochs):

    # FORWARD PASS: Least squares for rule consequents
    weights = firing_weights(X_train, centers, sigmas)

    A = design_matrix(X_train, weights)

    coef = np.linalg.lstsq(A, y_train, rcond=None)[0]
    coef = coef.reshape(nr, 4)

    # Individual rule outputs
    f = X_train @ coef[:,:3].T + coef[:,3]
```

```
# Final weighted output
pred = np.sum(weights*f, axis=1)

error = pred - y_train

errors.append(np.sqrt(np.mean(error**2)))

# BACKWARD PASS: Update MF centres and widths
grad_c = np.zeros_like(centers)
grad_s = np.zeros_like(sigmas)

for j in range(3):
  for k in range(3):

    dy_c = np.zeros(len(X_train))
    dy_s = np.zeros(len(X_train))

    for r, rule in enumerate(rules):

      if rule[j] == k:

        common = weights[:,r]*(f[:,r]-pred)

        dy_c += common*(X_train[:,j]-centers[j,k])/(sigmas[j,k]**2)

        dy_s += common*(X_train[:,j]-centers[j,k])**2/(sigmas[j,k]**3)

    grad_c[j,k] = 2*np.mean(error*dy_c)
    grad_s[j,k] = 2*np.mean(error*dy_s)
```

```
    # Gradient descent update
    centers = np.clip(centers-lr*grad_c, 0, 1)
    sigmas = np.clip(sigmas-lr*grad_s, 0.08, 0.6)


# Recalculate final consequent parameters after training
weights = firing_weights(X_train, centers, sigmas)
A = design_matrix(X_train, weights)

coef = np.linalg.lstsq(A, y_train, rcond=None)[0]
coef = coef.reshape(nr, 4)


# ============================================
# 7. PREDICTION FUNCTION
# ============================================

def anfis_predict(X):

    weights = firing_weights(X, centers, sigmas)

    # Linear output of every rule
    f = X @ coef[:,:3].T + coef[:,3]

    # Weighted average of rule outputs
    return np.sum(weights*f, axis=1)


# ============================================
# 8. TEST AND EVALUATE MODEL
# ============================================
```

```
y_pred = np.clip(anfis_predict(X_test), 0, 100)

MAE = np.mean(np.abs(y_test-y_pred))
RMSE = np.sqrt(np.mean((y_test-y_pred)**2))
R2 = 1 - np.sum((y_test-y_pred)**2)/np.sum((y_test-y_test.mean())**2)

print("\nANFIS PERFORMANCE")
print("MAE:", round(MAE, 3))
print("RMSE:", round(RMSE, 3))
print("R2 Score:", round(R2, 4))



# ============================================
# 9. BASELINE COMPARISON
# ============================================

# Baseline predicts the training-set mean every time
baseline = np.full(len(y_test), y_train.mean())

print("\nBASELINE PERFORMANCE")
print("MAE:", round(np.mean(np.abs(y_test-baseline)), 3))
print("RMSE:", round(np.sqrt(np.mean((y_test-baseline)**2)), 3))



# ============================================
# 10. LOW, MEDIUM, HIGH CLASSIFICATION
# ============================================

def category(score):
```

```
    if score < 33:
        return "Low"
    elif score < 66:
        return "Medium"
    else:
        return "High"


print("\nACTUAL VS PREDICTED")

for i in range(10):
    print(round(y_test[i],2),
          round(y_pred[i],2),
          category(y_pred[i]))


# ============================================
# 11. DISPLAY FUZZY RULE BASE
# ============================================

names = [
    ['Low','Medium','High'],
    ['Slow','Moderate','Fast'],
    ['Short','Moderate','Long']
]

print("\nFUZZY RULE BASE")

for i, (a,b,c) in enumerate(rules):

    p,q,r,s = coef[i]
```

```
    print("Rule", i+1,
          ": IF Density is", names[0][a],
          "AND Speed is", names[1][b],
          "AND Waiting is", names[2][c],
          "THEN Score =",
          np.round([p,q,r,s],2))



# ============================================
# 12. FIVE NEW TRAFFIC SCENARIOS
# ============================================

new = np.array([
    [10, 65, 10],
    [30, 45, 35],
    [55, 30, 70],
    [80, 15, 120],
    [95, 5, 170]
])

# Normalize using the same input ranges
new_scaled = new / np.array([100,80,180])

new_pred = np.clip(anfis_predict(new_scaled), 0, 100)

print("\nNEW SCENARIO PREDICTIONS")

for i in range(len(new)):
    print(new[i], "Score:", round(new_pred[i],2),
          "Category:", category(new_pred[i]))
```

```
# ============================================
# 13. PLOTS
# ============================================

# Training error
plt.figure(figsize=(6,4))
plt.plot(range(1,epochs+1), errors, marker='o')
plt.xlabel("Epoch")
plt.ylabel("Training RMSE")
plt.title("ANFIS Training Error")
plt.grid()
plt.show()

# Membership function plots
input_names = ['Vehicle Density', 'Average Speed', 'Waiting Time']
fuzzy_names = [
    ['Low','Medium','High'],
    ['Slow','Moderate','Fast'],
    ['Short','Moderate','Long']
]

plt.figure(figsize=(12,4))

for j in range(3):

    x = np.linspace(0,1,200)

    plt.subplot(1,3,j+1)
```

```
    for k in range(3):

        mu = np.exp(-0.5*((x-centers[j,k])/sigmas[j,k])**2)

        plt.plot(x,mu,label=fuzzy_names[j][k])

    plt.title(input_names[j])
    plt.xlabel("Normalized Input")
    plt.ylabel("Membership")
    plt.legend()
    plt.grid()

plt.tight_layout()
plt.show()

# Actual vs predicted
plt.figure(figsize=(6,5))
plt.scatter(y_test,y_pred)
plt.plot([0,100],[0,100],'r--')
plt.xlabel("Actual Congestion Score")
plt.ylabel("Predicted Congestion Score")
plt.title("Actual vs Predicted")
plt.grid()
plt.show()

print("\nANFIS EXPERIMENT COMPLETED")
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Membership functions
def tri(x, a, b, c):
   if x == b: return 1
   if x <= a or x >= c: return 0
   if x < b: return (x-a)/(b-a)
   return (c-x)/(c-b)

def trap(x, a, b, c, d):
   if b <= x <= c: return 1
   if a < x < b: return (x-a)/(b-a)
   if c < x < d: return (d-x)/(d-c)
   return 0

def gauss(x, mean, sigma):
   return np.exp(-((x-mean)**2)/(2*sigma**2))

# Universe
U = np.array([10, 15, 20, 25, 30, 35, 40])

# Fuzzy sets
cold = np.array([trap(x, 5, 10, 15, 25) for x in U])
warm = np.array([tri(x, 15, 25, 35) for x in U])
hot = np.array([trap(x, 25, 35, 40, 45) for x in U])
comfortable = gauss(U, 25, 5)

# Plot membership functions
for name, values in [('Cold', cold), ('Warm', warm),
                     ('Hot', hot), ('Comfortable', comfortable)]:
   plt.plot(U, values, marker='o', label=name)
```

```
plt.legend()
plt.show()

# Set operations
print("Union:", np.maximum(cold, warm))
print("Intersection:", np.minimum(cold, warm))
print("Complement:", 1 - warm)
print("Difference:", np.minimum(warm, 1-hot))
print("Algebraic product:", cold * warm)
print("Algebraic sum:", cold + warm - cold*warm)
print("Bounded sum:", np.minimum(1, cold + warm))
print("Bounded difference:", np.maximum(0, cold - warm))

# Properties of Warm
print("Support:", U[warm > 0])
print("Core:", U[warm == 1])
print("Height:", np.max(warm))
print("Cardinality:", np.sum(warm))
print("Alpha cut:", U[warm >= 0.5])
print("Normal:", np.max(warm) == 1)

convex = all(
   warm[i] >= min(warm[i-1], warm[i+1])
   for i in range(1, len(warm)-1)
)
print("Convex:", convex)

# Max-min and max-product composition
def compose(A, B, product=False):
   if product:
      values = A[:, :, None] * B[None, :, :]
```

```
    else:
        values = np.minimum(A[:, :, None], B[None, :, :])
    return np.max(values, axis=1)

R1 = np.array([
    [0.2,0.5,0.7,0.3],
    [0.7,0.6,0.2,0.1],
    [1.0,0.4,0.1,0.0],
    [0.3,0.6,0.8,0.4],
    [0.0,0.2,0.6,0.9]
])

R2 = np.array([
    [0.1,0.0,0.0,0.1],
    [0.5,0.3,0.2,0.2],
    [0.6,0.7,0.5,0.4],
    [0.4,0.6,0.9,0.2]
])

R3 = np.array([
    [0.8,0.4,0.1],
    [0.5,0.7,0.3],
    [0.6,0.5,0.2],
    [0.1,0.4,0.9]
])

print("R1 o R2 (max-min):\n", compose(R1, R2))
print("R1 o R2 (max-product):\n", compose(R1, R2, True))

print("Full chain (max-min):\n", compose(compose(R1,R2),R3))
print("Full chain (max-product):\n", compose(compose(R1,R2,True),R3,True))
```

```
# Max-min transitive closure
Q = np.array([
  [1,.8,.4,.2,.1],
  [.8,1,.9,.3,.2],
  [.4,.9,1,.5,.3],
  [.2,.3,.5,1,.7],
  [.1,.2,.3,.7,1]
])

while True:
  new_Q = np.maximum(Q, compose(Q, Q))
  if np.allclose(Q, new_Q):
    break
  Q = new_Q

print("Transitive closure:\n", Q)

# Extension principle: y = d^2 - 2d
D = [(-2,.3), (-1,.6), (0,1), (1,.8), (2,.5)]
result = {}

for d, mu in D:
  y = d**2 - 2*d
  result[y] = max(result.get(y, 0), mu)

print("Extension principle:", result)
```

2. Mamdani Fuzzy Logic Controller

```
import numpy as np
```

```
# Triangular membership function
def tri(x, a, b, c):
    if x == b: return 1
    if x <= a or x >= c: return 0
    if x < b: return (x-a)/(b-a)
    return (c-x)/(c-b)

# Negative, Zero, Positive memberships
def memberships(x, limit):
    return [
        tri(x, -limit, -limit, 0),
        tri(x, -limit, 0, limit),
        tri(x, 0, limit, limit)
    ]

# Mamdani controller
def controller(error, change):
    E = memberships(error, 10)
    D = memberships(change, 5)

    output = np.arange(-100, 101)

    # Cooling, Maintain, Heating
    output_sets = [
        np.array([tri(x, -100, -100, 0) for x in output]),
        np.array([tri(x, -100, 0, 100) for x in output]),
        np.array([tri(x, 0, 100, 100) for x in output])
    ]

    # Rules: rows = Error, columns = Change in error
```

```
    rules = [
        [0, 0, 1],
        [0, 1, 2],
        [1, 2, 2]
    ]

    aggregated = np.zeros(len(output))

    for i in range(3):
        for j in range(3):
            strength = min(E[i], D[j])
            rule_output = np.minimum(strength, output_sets[rules[i][j]])
            aggregated = np.maximum(aggregated, rule_output)

    # Centroid defuzzification
    return np.sum(output * aggregated) / np.sum(aggregated)

# Test cases
for error, change in [(8,0), (3,0), (0,0), (-3,0), (-8,0)]:
    print(error, change, "->", round(controller(error, change), 2))
```

3. ANFIS Traffic Congestion Prediction

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product

np.random.seed(42)

# 1. Generate dataset
n = 1200
```

```
D = np.random.uniform(0, 100, n)   # Density
S = np.random.uniform(0, 80, n)    # Speed
T = np.random.uniform(0, 180, n)   # Waiting time

W = 100 * T / 180
Vc = 100 * (1 - S / 80)

Y = 0.45*D + 0.35*W + 0.20*Vc

X = np.column_stack((D/100, S/80, T/180))

# Train-test split
idx = np.random.permutation(n)
train, test = idx[:960], idx[960:]

X_train, Y_train = X[train], Y[train]
X_test, Y_test = X[test], Y[test]

# 2. ANFIS setup: 2 Gaussian MFs per input = 8 rules
rules = list(product([0, 1], repeat=3))

centers = np.array([
   [0.25, 0.75],
   [0.25, 0.75],
   [0.25, 0.75]
], dtype=float)

sigma = 0.35

# 3. Calculate firing strengths and design matrix
def design(X, centers):
```

```
    m = [
        np.exp(-0.5*((X[:,i,None] - centers[i])/sigma)**2)
        for i in range(3)
    ]

    firing = np.column_stack([
        m[0][:,a] * m[1][:,b] * m[2][:,c]
        for a,b,c in rules
    ])

    normalized = firing / (firing.sum(axis=1, keepdims=True) + 1e-9)

    F = np.column_stack((X, np.ones(len(X))))

    A = np.hstack([
        normalized[:,i:i+1] * F
        for i in range(8)
    ])

    return firing, normalized, F, A

# 4. Hybrid learning
errors = []

for epoch in range(30):
    firing, normalized, F, A = design(X_train, centers)

    # Least squares: consequent parameters
    theta = np.linalg.lstsq(A, Y_train, rcond=None)[0]

    predicted = A @ theta
```

```
    errors.append(np.mean((Y_train - predicted)**2)**0.5)

    # Gradient descent: membership function centres
    rule_outputs = F @ theta.reshape(8, 4).T
    total = firing.sum(axis=1) + 1e-9
    new_centers = centers.copy()

    for d in range(3):
        for m in range(2):
            dh = np.column_stack([
                firing[:,r] * (X_train[:,d] - centers[d,m]) / sigma**2
                if rules[r][d] == m else np.zeros(len(X_train))
                for r in range(8)
            ])

            dy = np.sum(
                (rule_outputs - predicted[:,None]) * dh, axis=1
            ) / total

            gradient = np.mean(2 * (predicted - Y_train) * dy)
            new_centers[d,m] -= 0.01 * gradient

    centers = np.clip(new_centers, 0, 1)

# Recalculate final consequent parameters
_, _, _, A = design(X_train, centers)
theta = np.linalg.lstsq(A, Y_train, rcond=None)[0]

# 5. Test prediction
_, _, _, A_test = design(X_test, centers)
predicted = A_test @ theta
```

```
mae = np.mean(np.abs(Y_test - predicted))
rmse = np.sqrt(np.mean((Y_test - predicted)**2))
r2 = 1 - np.sum((Y_test-predicted)**2) / np.sum((Y_test-Y_test.mean())**2)

print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R2:", round(r2, 4))

# Baseline comparison: predict training mean
baseline = np.mean(Y_train)
print("Baseline RMSE:", round(np.sqrt(np.mean((Y_test-baseline)**2)), 2))

# 6. Congestion categories
def category(score):
    if score < 33:
        return "Low"
    elif score < 66:
        return "Medium"
    return "High"

print("\nTest predictions:")
for actual, pred in zip(Y_test[:5], predicted[:5]):
    print(round(actual,2), round(pred,2), category(pred))

# 7. Five new traffic scenarios
new = np.array([
    [10,65,10],
    [30,45,35],
    [55,30,70],
    [80,15,120],
```

```
    [95,5,170]
])

new_X = np.column_stack((
    new[:,0]/100,
    new[:,1]/80,
    new[:,2]/180
))

_, _, _, A_new = design(new_X, centers)
new_predictions = A_new @ theta

print("\nNew scenarios:")
for row, pred in zip(new, new_predictions):
    print(row, "Score:", round(pred,2), "Category:", category(pred))

# 8. Required plots
plt.figure(figsize=(12,4))

for i in range(3):
    plt.subplot(1,3,i+1)
    x = np.linspace(0,1,100)
    for c in centers[i]:
        plt.plot(x, np.exp(-0.5*((x-c)/sigma)**2))
    plt.title(["Density","Speed","Waiting Time"][i])

plt.tight_layout()
plt.show()

plt.plot(errors)
plt.xlabel("Epoch")
```

```
plt.ylabel("Training RMSE")
plt.title("Training Error")
plt.show()

plt.scatter(Y_test, predicted)
plt.xlabel("Actual Score")
plt.ylabel("Predicted Score")
plt.title("Actual vs Predicted")
plt.show()
```

1. Implement a Simple Intelligent Agent Using Python

```
# Environment
rooms = {"A": "Dirty", "B": "Dirty"}
location = "A"

# Intelligent agent
def agent(location, rooms):
    if rooms[location] == "Dirty":
        return "Suck"
    elif rooms["A"] == "Dirty":
        return "Left"
    elif rooms["B"] == "Dirty":
        return "Right"
    else:
        return "Stop"

# Run the agent
while True:
    action = agent(location, rooms)
    print("Location:", location, "Action:", action)
```

```
    if action == "Stop":
        break
    elif action == "Suck":
        rooms[location] = "Clean"
    elif action == "Right":
        location = "B"
    elif action == "Left":
        location = "A"

print("Final:", rooms)
```

2. Design a Simple Expert System

```
# Facts
facts = {
    "no_power": True,
    "battery_low": True,
    "computer_slow": False
}

# Knowledge base: IF conditions THEN solution
rules = [
    ({"no_power": True, "battery_low": True},
     "Charge the battery."),

    ({"no_power": True, "battery_low": False},
     "Check the power cable."),

    ({"computer_slow": True},
     "Close unnecessary background applications.")
]
```

```
# Inference engine
for conditions, solution in rules:
    if all(facts.get(k) == v for k, v in conditions.items()):
        print("Advice:", solution)\\
```

3. ANN for Half Adder and Full Adder

```
import numpy as np
from itertools import product

# Half Adder truth table
X_half = np.array(list(product([0, 1], repeat=2)))
Y_half = np.array([[a ^ b, a & b] for a, b in X_half])

# Full Adder truth table
X_full = np.array(list(product([0, 1], repeat=3)))
Y_full = np.array([
    [a ^ b ^ c, int(a+b+c >= 2)]
    for a, b, c in X_full
])

# Activation functions
def activate(x, name):
    if name == "sigmoid":
        return 1 / (1 + np.exp(-np.clip(x, -30, 30)))
    if name == "tanh":
        return np.tanh(x)
    if name == "relu":
        return np.maximum(0, x)
    if name == "leaky":
```

```
        return np.maximum(0.01*x, x)

def derivative(x, name):
    if name == "sigmoid":
        y = activate(x, name)
        return y * (1-y)
    if name == "tanh":
        y = np.tanh(x)
        return 1-y*y
    if name == "relu":
        return (x > 0).astype(float)
    if name == "leaky":
        return np.where(x > 0, 1, 0.01)

# ANN training
def train(X, Y, name):
    np.random.seed(42)

    W1 = np.random.randn(X.shape[1], 8) * 0.7
    b1 = np.ones((1, 8)) * 0.2
    W2 = np.random.randn(8, 2) * 0.7
    b2 = np.zeros((1, 2))

    for epoch in range(10000):
        # Forward propagation
        z1 = X @ W1 + b1
        h = activate(z1, name)

        z2 = h @ W2 + b2
        out = activate(z2, "sigmoid")
```

```
        # Backpropagation
        error = out - Y
        d2 = error * out * (1-out)
        d1 = (d2 @ W2.T) * derivative(z1, name)

        W2 -= 0.1 * (h.T @ d2) / len(X)
        b2 -= 0.1 * np.mean(d2, axis=0, keepdims=True)
        W1 -= 0.1 * (X.T @ d1) / len(X)
        b1 -= 0.1 * np.mean(d1, axis=0, keepdims=True)

    return (out >= 0.5).astype(int)

# Compare activation functions
for name in ["sigmoid", "tanh", "relu", "leaky"]:
    print("\nActivation:", name)

    half_output = train(X_half, Y_half, name)
    full_output = train(X_full, Y_full, name)

    print("Half Adder:", np.array_equal(half_output, Y_half))
    print("Full Adder:", np.array_equal(full_output, Y_full))

    print("Half Adder Output:\n", half_output)
    print("Full Adder Output:\n", full_output)
\\\\\\\\\\\\\\\\\\\\\\\\\\\\
x = np.array([-2, -1, 0, 1, 2])

print("Sigmoid:", 1/(1+np.exp(-x)))
print("Tanh:", np.tanh(x))
print("ReLU:", np.maximum(0, x))
print("Leaky ReLU:", np.maximum(0.01*x, x))
```

```
e = np.exp(x - np.max(x))

print("Softmax:", e / np.sum(e))
```
